

TP 4. Uso de información contenida en alineamientos múltiples (Parte I): motivos lineales, expresiones regulares, PROSITE

Objetivos

1. Realizar un alineamiento múltiple de secuencias
2. Aprender la simbología de las expresiones regulares y cómo se utilizan para representar motivos.
3. Construir una expresión regular a partir de evidencia experimental.

Alineamientos múltiples

Un alineamiento múltiple (MSA) involucra tres o más secuencias biológicas. Debido a que la tarea de alinear múltiples secuencias de largos biológicamente significativos suele ser muy demandante en términos de recursos computacionales y tiempos de ejecución estos requieren metodologías más sofisticadas para llevarse a cabo. Por ello la mayoría de los programas disponibles para realizar MSA utiliza heurísticas en vez de algoritmos de optimización global.

Heurística

Es una estrategia que busca resolver un problema más simple cuya solución se interseca con la solución de un problema más complejo. Generalmente esto implica que no es seguro encontrar el mejor resultado pero sí una solución que sea aceptable. Las heurísticas se aplican con frecuencia en computación para poder resolver problemas que, por su complejidad, serían imposibles de abordar dados los limitados recursos con los que se cuentan.

Dadas las secuencias de aminoácidos de un set de proteínas que se quieren comparar, el MSA muestra los residuos de cada proteína en una fila junto con los gaps que le correspondan de tal manera que todos los residuos "equivalentes" se encuentren en la misma columna. La utilidad de esta equivalencia depende de quien mire el alineamiento:

- Alguien que hace una filogenia puede enfocarse en que comparten un ancestro común;
- Alguien que hace biología estructural puede enfocarse en que son residuos en posiciones análogas de una estructura proteica;
- Alguien que hace biología molecular puede enfocarse en el rol funcional de esos residuos en la proteína.

En cada caso un MSA provee un pantallazo sobre las restricciones evolutivas, estructurales o funcionales que caracterizan un set de proteínas de una manera visual e intuitiva.

Un pipeline típico para realizar un MSA sería:

1. Formular la pregunta que se quiere contestar. Por ejemplo, "¿Qué estructura secundaria adopta X región de mi proteína de interés?"
2. Obtener secuencias que puedan contestar a mi pregunta. Por ejemplo, secuencias que estén relacionadas a mi proteína de interés.
3. Utilizar alguno de los programas disponibles para llevar a cabo el MSA. Por ej. EMBOSS
4. Realizar ajustes manuales para corregir posibles errores de los algoritmos de alineamiento.

Algoritmos de alineamiento múltiple: MUSCLE, Clustal y T-Coffee

Antes de meternos con los patrones derivados de un MSA, vale la pena entender cómo se construye ese MSA en primer lugar. Ninguno de estos tres programas prueba todas las combinaciones posibles de alinear N secuencias (el problema es NP-completo y se vuelve intratable ya con pocas secuencias); todos usan **heurísticas** para llegar a una solución buena, aunque no necesariamente óptima, en tiempo razonable. Donde se diferencian es en qué heurística usan y qué compromiso hacen entre velocidad y precisión.

ClustalW / Clustal Omega

Es el método **progresivo** clásico, y probablemente el más usado históricamente. Funciona en tres etapas:

1. Calcula todas las alineaciones de a pares (pairwise) entre las secuencias de entrada.
2. Con esas distancias, construye un **árbol guía** (guide tree) que aproxima la relación evolutiva entre las secuencias.

3. Va alineando las secuencias de a una, siguiendo el orden del árbol: primero las más parecidas entre sí, y después va sumando las más lejanas al alineamiento ya construido.

El problema clásico de este enfoque ("greedy" o voraz) es que si se comete un error en las primeras alineaciones (entre las secuencias más similares), ese error queda "congelado" y se arrastra al resto del proceso — no hay forma de corregirlo más adelante. **Clustal Omega** es la evolución moderna de ClustalW: en vez de alinear secuencia contra secuencia, usa perfiles basados en modelos ocultos de Markov (HMM), lo que lo hace mucho más escalable a miles de secuencias sin perder demasiada precisión.

MUSCLE

MUSCLE (**M**Ultiple **S**equences **C**omparison by **L**og-**E**xpectation) también es progresivo en su núcleo, pero agrega dos ideas para mejorar sobre ClustalW:

1. Para armar el árbol guía inicial usa una estimación de distancia muy rápida basada en conteo de k-mers (subsecuencias cortas compartidas), en vez de hacer todas las alineaciones de a pares — esto lo hace mucho más rápido en la etapa inicial.
 2. Después de la alineación progresiva, aplica una etapa de **refinamiento iterativo**: recorre el árbol dividiéndolo en dos particiones repetidas veces, y si separar y realinear esas dos mitades mejora el score global, actualiza el alineamiento.
- El resultado es un método diseñado explícitamente para ser rápido con conjuntos grandes de secuencias, manteniendo una precisión competitiva con métodos más lentos.

T-Coffee

T-Coffee (**T**ree-based **C**onsistency **O**bjective **F**unction for alignment **E**valuation) prioriza la precisión por sobre la velocidad, usando un enfoque distinto: en vez de construir el alineamiento final basándose solo en un árbol guía y alinear de a pares en ese orden, primero arma una **"librería de alineamientos"** combinando información de alineamientos globales (tipo ClustalW) y locales (tipo Lalign) entre todos los pares de secuencias. Después, al construir el alineamiento múltiple progresivo, en cada paso consulta esa librería para maximizar la **consistencia**: privilegia las relaciones entre residuos que aparecen respaldadas por múltiples alineamientos de a pares, no solo por la comparación directa entre las dos secuencias que se están combinando en ese momento. Esto reduce el efecto de "error temprano que se arrastra" que tiene ClustalW, a costa de ser considerablemente más lento (la construcción de la librería es costosa computacionalmente).

Tabla comparativa

	Clustal (W / Omega)	MUSCLE	T-Coffee
Estrategia base	Progresiva	Progresiva con refinamiento iterativo	Progresiva basada en consistencia
Cómo arma el árbol guía	Alineamientos de a pares (ClustalW) / perfiles HMM (Omega)	Distancia rápida por k-mers, luego refinada	Librería de alineamientos globales + locales entre todos los pares
Corrige errores tempranos	No (Omega mejora escalabilidad, no esto)	Parcialmente, vía refinamiento iterativo post-alineamiento	Sí, es su objetivo de diseño principal
Velocidad	Alta (Omega especialmente, apto para miles de secuencias)	Muy alta — de los más rápidos con datasets grandes	Baja/media — es el más lento de los tres
Precisión típica	Buena, algo por debajo de los otros dos en benchmarks	Alta, comparable a T-Coffee en muchos benchmarks	Alta — frecuentemente el mejor score en benchmarks como BALiBASE
Mejor caso de uso	Muchas secuencias, se prioriza velocidad/escala	Buen balance velocidad/precisión, datasets grandes	Pocas secuencias, divergentes, donde la precisión importa más que el tiempo

Ejercicio 1

La gp120 es una proteína que recubre al virus del HIV y facilita su unión e ingreso a la célula que infecta (linfocitos CD4+)

Entre nuestros archivos contamos con un multifasta (gp120.fasta) que contiene 27 secuencias de gp120 de HIV-1, HIV-2 y SIV.

Estas proteínas contienen 9 puentes disulfuro conservados. También es de interés el loop V3, una porción expuesta de la proteína, conocido target de anticuerpos el cual constituye una región hipervariable dada la presión selectiva a la que se ve sometido.

Pueden ver la disposición de las distintas regiones de la gp120 en el siguiente link:

Paso 0:

Genera el entorno de trabajo y descarga la secuencia gp120.

```
wget "https://raw.githubusercontent.com/mercedesgarnham/Tutoriales/refs/heads/main/TP4/data/gp120.fasta"
```

Paso 1:

Utiliza las herramientas de EMBOSS para realizar un alineamiento múltiple con las secuencias de gp120 (recuerden que para buscar herramientas pueden usar `wossname`)

El comando a utilizar es `emma` . Para ver la ayuda, tipea `emma -help` en la terminal.

Paso 2:

Ingresa a <https://alignmentviewer.org/> y carga el archivo `hv1a2.aln` para visualizar el alineamiento.

Paso 3:

Utiliza el comando `showalign` de EMBOSS para obtener una mejor visualización del alineamiento.

Paso 4:

Observa el alineamiento, como primer control podemos corroborar que las 18 Cisteínas (**C**) estén bien alineadas.

Paso 5:

Utiliza el esquema de gp120 para identificar diversas regiones ya sea conservadas o muy variables (Estructuras, loops, etc.)

Nota que las posiciones en el alineamiento cuentan gaps por lo que no se corresponden exactamente con el esquema. Utiliza las posiciones de las cisteínas conservadas para identificar diferentes regiones.

Paso extra:

Para visualizar el árbol obtenido pueden ingresar a <https://itol.embl.de/> y cargar el archivo `hv1a2.dnd`

Introducción a los motivos lineales y expresiones regulares

Motivos lineales (SLiMs)

Los motivos lineales (**Short Linear Motifs**) son segmentos de secuencia de entre 3 y 15 aminoácidos que cumplen funciones específicas, como la unión a otras proteínas, la localización subcelular, la fosforilación o la degradación. A diferencia de los dominios globulares, los SLiMs suelen encontrarse en regiones desordenadas de las proteínas y son muy degenerados: toleran sustituciones de aminoácidos en varias posiciones, lo que permite una rápida evolución y regulación.

Expresiones regulares (Regex)

Una expresión regular es una cadena de caracteres que describe un patrón de búsqueda. En bioinformática, se utilizan para representar motivos de secuencia de forma compacta. La sintaxis más común es la empleada por **PROSITE** y muchas herramientas de búsqueda de motivos.

Símbolo	Significado
.	Cualquier aminoácido (o nucleótido) es permitido.
[XY]	Solo los aminoácidos X e Y son permitidos.
[^XY]	Los aminoácidos X e Y están prohibidos.
{min,max}	Número mínimo y máximo de veces que se puede repetir una posición (ej. $x\{2,3\}$).
^X	El aminoácido X se encuentra en el extremo N-terminal.
X\$	El aminoácido X se encuentra en el extremo C-terminal.
A B	Se encuentra, o bien el aminoácido A, o bien el B (alternancia).
(AB) (CD)	Se encuentra la secuencia AB o la secuencia CD.

Las expresiones regulares pueden combinarse para describir motivos más complejos. Por ejemplo, el motivo de unión a Rev1 que veremos más adelante se representa como `FF[^P]{0,2}[KR]{1,2}[^P]{0,4}` .

Ejercicio 2: Construcción de expresiones regulares a partir de ejemplos de motivos conocidos

En este ejercicio, vamos a practicar la construcción de expresiones regulares para diferentes motivos lineales descritos en la literatura. Para cada motivo, se proporcionan ejemplos de secuencias que lo cumplen. Tu tarea es deducir la expresión regular que los representa.

Motivo 1: DOC_CYCLIN_RxL_1

Descripción: Motivo de anclaje (docking) reconocido por los complejos Ciclina/CDK de la fase S en hongos y mamíferos. La expresión regular canónica es `[RK].L.{0,1}[FYLIIVMP]` .

 Pregunta:

Indicar si estas secuencias cumplen el motivo:

- RMLP
- EKSLTSP
- KSLGI

Motivo 2: LIG_ARL_BART_1

Descripción: Motivo presente en el extremo N-terminal de las proteínas ARL2 y ARL3 que asegura la unión dependiente de GTD a BART y BARTL1. La expresión regular es `^..LL[^P]IL[^P][^P][LM]` .

 Pregunta:

Indicar si estas secuencias cumplen el motivo:

- MLLLDIELDL
- LALLPIIEDM
- LALLGIIEDM

Motivo 3: DOC_PP2A_B56_1

Descripción: Sitio de anclaje requerido por la subunidad reguladora B56 de la fosfatasa PP2A para la desfosforilación de proteínas. La expresión regular es `([LMFYWIC]..I.E) | (L..[IVLWIC].E)` .

 Pregunta:

Indicar si estas secuencias cumplen el motivo:

- MLPAIA
- MLPAIAE
- MLPAAE

Ejercicio 3: Reconocimiento de secuencias que cumplen un patrón

Resolver:

Dada la siguiente expresión regular:

`[RK] . {0, 1} [P] . {2} [^P]`

Indicar si estas secuencias cumplen el motivo:

- A. RPAVL
- B. KAPLRP
- C. RRPAPP
- D. RRP GPA
- E. RPPGA

Ejercicio 4: Construcción de una expresión regular a partir de evidencia experimental

Vamos a trabajar con el motivo de unión a la proteína **Rev1**, que participa en la reparación del ADN por Translesion Synthesis (TLS). La evidencia experimental (obtenida de estudios estructurales y de mutagénesis) indica que:

- El motivo está mediado por **dos fenilalaninas consecutivas** (FF).
- Estas fenilalaninas son seguidas por una región de **0 a 2 aminoácidos** que no pueden ser prolina (P).
- Luego hay **1 o 2 aminoácidos** con carga positiva (lisina K o arginina R).
- Finalmente, una región de **0 a 4 aminoácidos** que no pueden ser prolina (P).

A continuación se muestran fragmentos de secuencia de proteínas que se unen a Rev1 (la interacción fue verificada experimentalmente):

```
>sp|Q03834|MSH6_YEAST|31-38
SQKKMKQSSLSFFSKQVPSGTPSKKVQ
```

```
>sp|Q60596|XRCC1_MOUSE|191-200
DDSANSLKPGALFFSRINKTSSASTSDPAG
```

```
>sp|Q9H040|SPRTN_HUMAN|418-428
RPRLEDKTVFDNFFIKKEQIKSSGNDPKYST
```

```
>sp|Q15054|DPOD3_HUMAN|236-245
NKAPGKGNMMSNFFGKAAMNFKVNLDSQ
```

```
>sp|Q9UNA4|POLI_HUMAN|569-579
SCPLHASRGVLSFFSKKQMDIPINPRDHL
```

```
>sp|Q9Y253|POLH_HUMAN|481-490
TATKKATTSLFFQKAAERQKVKEASLSS
```

```
>sp|Q9QUG2|POLK_MOUSE|564-575
LAKPLEMSHKKSFFDKRSERISNCQDTSRCK
```

Paso 1: Identificar el motivo común

1. Genera el motivo que cumple con lo encontrado por evidencia experimental.
2. Copia las secuencias anteriores y pégalas en la herramienta **regex101** (<https://regex101.com>) en el recuadro "Test String".

3. En el campo "Regular Expression", escribe una expresión regular que capture el motivo descrito por la evidencia experimental. Recuerda utilizar la sintaxis.
4. Verifica que tu expresión regular encuentre todas las secuencias del listado (debería haber al menos una coincidencia por cada fragmento).

Paso 2: Evaluar una secuencia adicional

1. Ahora prueba si la siguiente secuencia (de la levadura *Saccharomyces cerevisiae*) cumple con el motivo:

sp | Q04049 | POLH_YEAST | 625-632
KKQVTSSKNILSFFTRKK

2. Pégalala en regex101 (añádela al "Test String") y observa si tu expresión regular la reconoce.

Introducción: de un MSA a un patrón

¿Cómo nace un patrón de PROSITE?

Cuando varias proteínas relacionadas comparten una función bioquímica puntual (por ejemplo, todas son fosforiladas por la misma quinasa, o todas unen ATP de la misma manera), sus secuencias alrededor de ese sitio funcional suelen alinearse en una columna conservada dentro de un MSA. Al observar esa región en el alineamiento:

- Las posiciones **invariantes** (el mismo aminoácido en todas las secuencias) se escriben como letras fijas.
- Las posiciones **parcialmente conservadas** (un grupo reducido de aminoácidos posibles) se escriben como clases [XYZ] .
- Las posiciones sin conservación relevante se escriben como x (equivalente al . que venimos usando).
- Las posiciones que **nunca** aparecen (porque bioquímicamente impiden la función, como una prolina que rompe una hélice) se escriben como {X} (equivalente a [^X]).

El resultado es una expresión regular generalizada: el patrón de PROSITE. La base de datos no guarda "una secuencia ejemplo", guarda la regla derivada del alineamiento de todas las instancias conocidas.

Ejercicio 5: Leer patrones reales de PROSITE

La siguiente tabla muestra patrones reales tal como figuran en la base de datos PROSITE, con su sintaxis original (con guiones) y su equivalente en regex de estilo regex101 (con puntos):

ID PROSITE	Nombre	¿Qué es?	Patrón original (PROSITE)	Regex equivalente
PS00001	Sitio de N-glicosilación	Marca dónde una proteína puede recibir un azúcar (oligosacárido) unido al nitrógeno de una Asparagina (N), durante su paso por el retículo endoplasmático/Golgi. Es una de las modificaciones post-traduccionales más comunes en proteínas de membrana y secretadas.	N - {P} - [ST] - {P}	N [^ P] [ST] [^ P]
PS00005	Sitio de fosforilación por PKC	PKC (Proteína Quinasa C) es una familia de enzimas quinasas que fosforilan residuos de Serina o Treonina en proteínas blanco, activadas típicamente por diacilglicerol (DAG) y Ca ²⁺ . Es central en vías de transducción de señales que regulan crecimiento celular, proliferación y apoptosis.	[ST] - x - [RK]	[ST] . [RK]
PS00006	Sitio de fosforilación por caseína quinasa II	CK2 (Caseína Quinasa II) es otra serina/treonina quinasa, constitutivamente activa (no depende de segundos mensajeros como PKC), que fosforila cientos de proteínas involucradas en control del ciclo celular, transcripción y respuesta al estrés.	[ST] - x (2) - [DE]	[ST] . { 2 } [DE]

ID PROSITE	Nombre	¿Qué es?	Patrón original (PROSITE)	Regex equivalente
PS00009	Sitio de amidación	La amidación C-terminal es una modificación en la que el extremo C-terminal de una proteína (normalmente un ácido carboxílico) se convierte en una amida. Es un paso de maduración esencial para muchas hormonas peptídicas y neuropéptidos (por ejemplo, oxitocina, calcitonina): sin la amidación, el péptido pierde actividad biológica.	x - G - [RK] - [RK]	. G[RK][RK]
PS00017	Motivo A de unión a ATP/GTP (P-loop, Walker A)	El P-loop (Walker A) es un motivo estructural (un bucle rico en Glicinas que envuelve los fosfatos del nucleótido) presente en una enorme variedad de proteínas que unen ATP o GTP: quinasas, ATPasas, motores moleculares (miosina, kinesina), proteínas G, etc. Es uno de los motivos lineales más antiguos y conservados evolutivamente.	[AG] - x (4) - G - K - [ST]	[AG] . {4}GK[ST]

 Resolver

Para cada uno de los 5 patrones de la tabla, proponé (a mano, sin herramientas) una secuencia corta de aminoácidos que lo cumpla. Después verificá cada una en [regex101](#) usando la columna "Regex equivalente".

Escanear una proteína con ScanPrositate

Vamos a retomar la p53 humana (UniProt **P04637**) enfocándonos en los patrones de fosforilación de la tabla anterior.

 Pasos a seguir :

- Entrá a [ScanPrositate](#).
- Pegá la secuencia de p53 (UniProt P04637) en formato FASTA.
- Antes de ejecutar, **destildá** la opción "Exclude motifs with a high probability of occurrence" (por defecto está tildada y oculta justamente los patrones cortos y degenerados como los que vamos a buscar).
- Ejecutá la búsqueda y, en la lista de resultados, buscá específicamente si aparecen hits para **PS00005** (PKC) y **PS00006** (caseína quinasa II).

¿Cuál es "el residuo" en cada hit?

En ambos patrones (PS00005: [ST].[RK] y PS00006: [ST].[2]{DE}), el residuo que se fosforila es siempre la **primera posición** del match: la S o la T inicial. Las posiciones restantes del patrón no se modifican — son el contexto de secuencia que la quinasa reconoce para fosforilar esa S/T. Por ejemplo, si ScanPrositate te muestra un hit "99-101: SqK", el residuo fosforilable es la **Serina en la posición 99** (S99); la "q" (minúscula, cualquier aminoácido) y la "K" en 101 no se fosforilan, solo marcan el contexto del sitio.

 Pregunta 2:

¿Cuántos hits obtuviste para PS00005 y para PS00006? Elegí uno de esos hits y anotá el número de posición y la identidad (S o T) del residuo fosforilable (la primera posición del patrón, como se explicó arriba).

 Paso 3:

Elegí uno de los hits de PS00005 o PS00006 que encontraste en p53. Buscá en UniProt si esa posición corresponde a un sitio de fosforilación experimentalmente confirmado y conservado entre ortólogos de p53 en otras especies (podés revisar la sección "PTM/Processing" de la entrada P04637 en UniProt).

- ¿El sitio que elegiste está anotado como sitio de fosforilación real en UniProt?
- ¿Qué te dice esto sobre la relación entre "cumplir un patrón de PROSITE" y "ser un sitio funcional verificado"?

Ejercicio 6: De un motivo a las proteínas que lo cumplen

Hasta ahora fuimos siempre en una dirección: tenías una proteína y buscabas qué motivos contiene. Ahora vamos al revés: tenés un motivo (una regex) y buscás **qué proteínas, en toda una base de datos, lo cumplen**. Esto es exactamente lo que hace ELM cuando arma su catálogo, y es

la otra mitad de la funcionalidad de ScanProsite (el modo "Pattern/motif search" en vez de "Protein scan").

Las dos direcciones de ScanProsite

- **Proteína** → **motivos** (lo que hiciste en el Ejercicio 5 y el Paso 2 de este ejercicio extra): das una secuencia, te devuelve qué patrones de PROSITE cumple.
- **Motivo** → **proteínas** (lo que vamos a hacer ahora): das un patrón (propio o de PROSITE), y ScanProsite escanea una base de datos completa (UniProtKB/Swiss-Prot, PDB, etc.) devolviendo todas las proteínas que lo cumplen.

Esta segunda dirección es la que se usa para *descubrir* candidatos nuevos: por ejemplo, para encontrar proteínas humanas no descriptas todavía que podrían unir ATP porque tienen el P-loop, o proteínas que podrían interactuar con Rev1 porque tienen su motivo de anclaje.

 Paso 1: Buscar todas las proteínas humanas con el P-loop

1. Entrá de nuevo a [ScanProsite](#).
2. Buscá en la página el formulario para ingresar un **patrón** en vez de una secuencia (en la interfaz suele estar como una pestaña o sección separada, distinta del cuadro de "Test String"/secuencia).
3. Ingresá el patrón del motivo A de unión a ATP/GTP (P-loop, PS00017) en sintaxis PROSITE: `[AG]-x(4)-G-K-[ST]`
4. Restringí la búsqueda a la base de datos **UniProtKB/Swiss-Prot**.
5. Ejecutá la búsqueda.

 Pregunta 5:

¿Cuántas proteínas humanas revisadas (Swiss-Prot) devolvió la búsqueda?
Elegí 3 al azar de la lista y fijate en su nombre/función en UniProt: ¿son consistentes con unión de nucleótidos (quinasas, ATPasas, GTPasas, motores moleculares, etc.)?

Cierre del Tutorial

Completa la siguiente tabla para resumir lo visto:

Tema	Herramienta(s) utilizada(s)	Tipo de datos	Análisis del resultado
Expresiones regulares	(completar)	(completar)	(completar)
Búsqueda de motivos en bases de datos	(completar)	(completar)	(completar)
Descubrimiento de motivos <i>de novo</i>	(completar)	(completar)	(completar)

Objetivos

- Realizar un alineamiento múltiple de secuencias. (Sí / No)
- Aprender la simbología de las expresiones regulares y cómo se utilizan para representar motivos. (Sí / No)
- Construir una expresión regular a partir de evidencia experimental. (Sí / No)

Bibliografía complementaria

- PROSITE documentation: <https://prosite.expasy.org/>
- Dinkel H, Van Roey K, Michael S, et al. (2016) ELM 2016—data update and new functionality of the eukaryotic linear motif resource. Nucleic Acids Research 44(D1): D294-D300.